Solution: Maximum Length of Pair Chain

Problem Statement

Given a collection of pairs where each pair contains two elements [a, b], find the maximum length of a chain you can form using pairs.

A pair [a, b] can follow another pair [c, d] in the chain if b < c.

You can select pairs in any order and don't need to use all the given pairs.

Examples

1. **Example 1:**
    - **Input:** `[[1,2], [3,4], [2,3]]`
    - **Expected Output:** `2`
    - **Justification:** The longest chain is `[1,2] -> [3,4]`. The chain `[1,2] -> [2,3]` is invalid because 2 is not smaller than 2.
2. **Example 2:**
    - **Input:** `[[5,6], [1,2], [8,9], [2,3]]`
    - **Expected Output:** `3`
    - **Justification:** The chain can be `[1,2] -> [5,6] -> [8,9]` or `[2,3] -> [5,6] -> [8, 9]`.
3. **Example 3:**
    - **Input:** `[[7,8], [5,6], [1,2], [3,5], [4,5], [2,3]]`
    - **Expected Output:** `3`
    - **Justification:** The longest possible chain is formed by chaining `[1,2] -> [3,5] -> [7,8]`.

Solution

The greedy approach to solving the problem involves initially sorting the pairs based on their second elements. This step is crucial as it aligns the pairs in a way that the one with the smallest end is considered first, leading to more opportunities for chain extension.

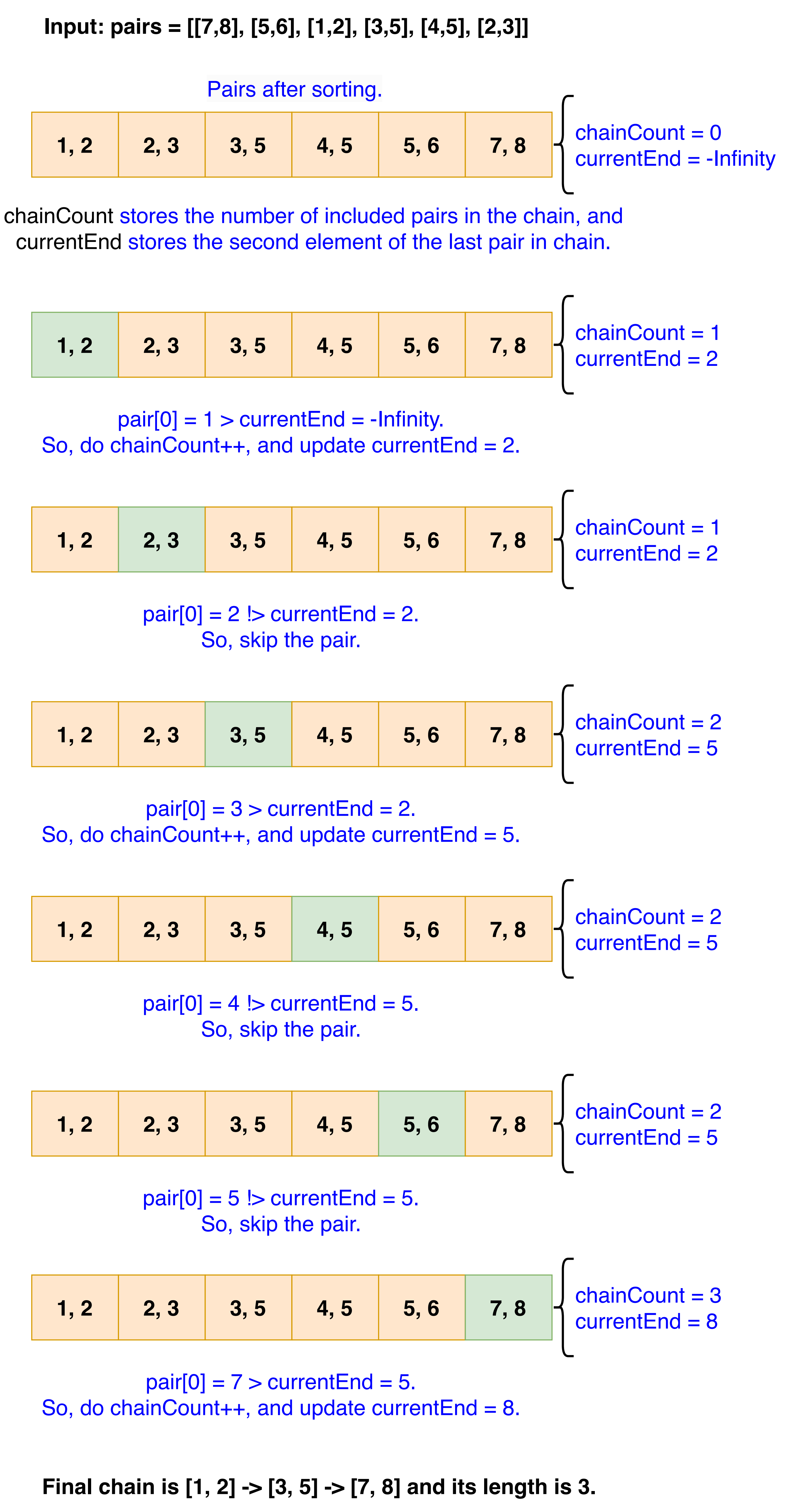
After sorting, we iterate through the pairs, maintaining a variable to track the current end of the chain. For each pair, if the first element is greater than the current chain end, we extend the chain by adding this pair and updating the chain end to this pair's second element. This method ensures that at each step, we're making the most optimal choice to extend the chain without needing to consider previous pairs, thereby maximizing the number of pairs in the chain with the least end values first, leading to the longest possible chain.

1. **Sorting the Pairs:** Initially, sort all pairs based on their second element in ascending order. This ensures that as you iterate through the pairs, you are always considering the pair with the next smallest endpoint.
2. **Initializing Variables:** Start with two variables: one to keep track of the current endpoint of the chain (`currentEnd`) and another to count the number of pairs in the chain (`chainCount`). Initialize `currentEnd` to the lowest possible value (e.g., Integer.MIN_VALUE) and `chainCount` to 0.
3. **Iterating and Choosing Pairs:** Iterate through the sorted pairs. For each pair, check if its first element is greater than `currentEnd`. If it is, it means this pair can be appended to the current chain. Update `currentEnd` to the second element of this pair and increment `chainCount`.
4. **Result:** After iterating through all pairs, `chainCount` will hold the maximum number of pairs that can be chained.

This Greedy approach is effective because it always chooses the option that seems best at the moment (the pair with the smallest endpoint) and this local optimal choice leads to a globally optimal solution in this specific problem context. The logic behind this is that by choosing the pair with the smallest endpoint, you are maximizing the potential for other pairs to be chained afterward.

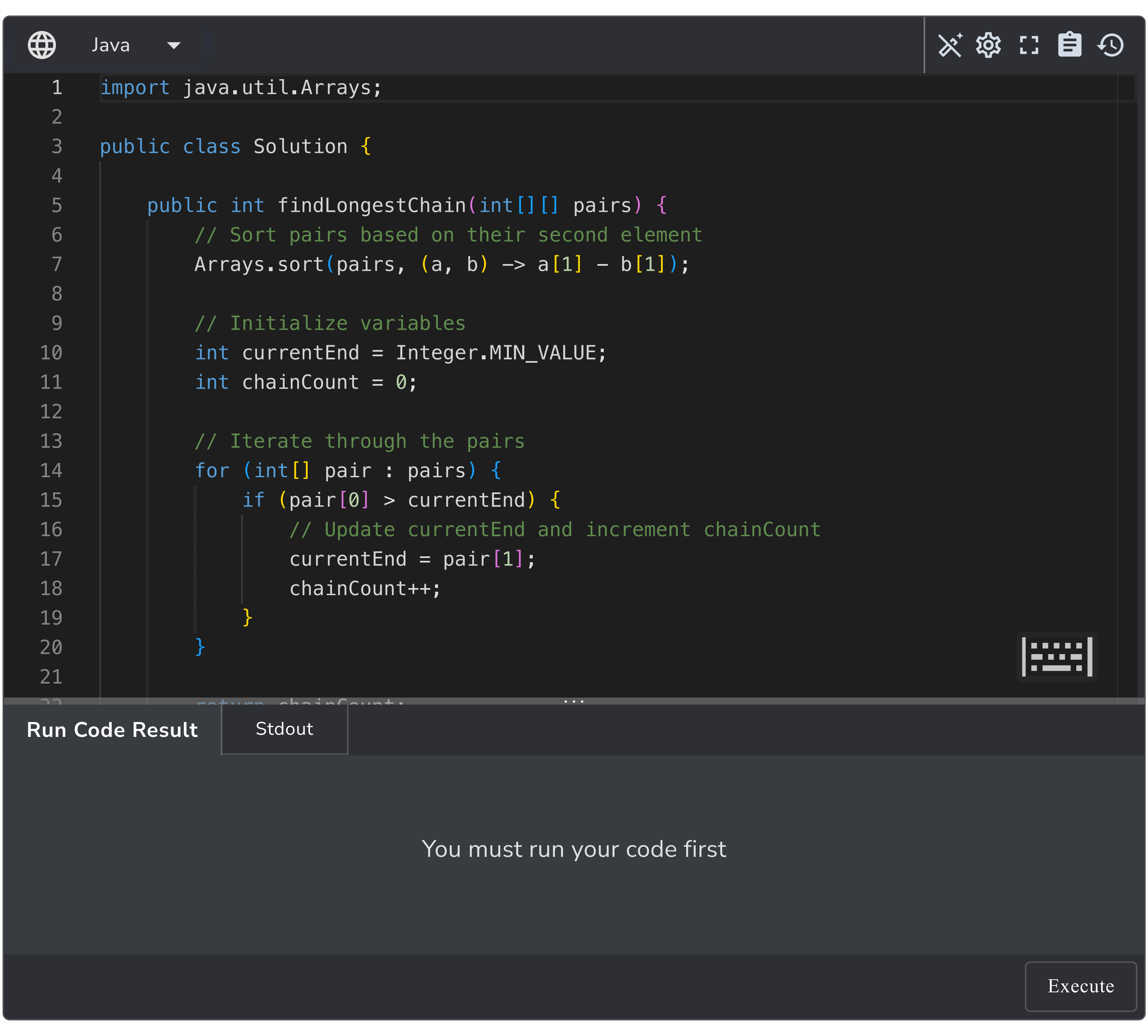
Algorithm Walkthrough

1. **Input Pairs:** `[[7,8], [5,6], [1,2], [3,5], [4,5], [2,3]]`
2. **After Sorting by Second Element:** `[[1,2], [2,3], [3,5], [4,5], [5,6], [7,8]]`
3. **Iterating through Pairs:**
    - Start with `currentEnd = Integer.MIN_VALUE` and `chainCount = 0`.
    - Pair `[1,2]`: `1 > Integer.MIN_VALUE`. Update `currentEnd` to `2`, `chainCount` to `1`.
    - Pair `[2,3]`: `2 > 2` is false. Skip.
    - Pair `[3,5]`: `3 > 2`. Update `currentEnd` to `5`, `chainCount` to `2`.
    - Pair `[4,5]`: `4 > 5` is false. Skip.
    - Pair `[5,6]`: `5 > 5` is false. Skip.
    - Pair `[7,8]`: `7 > 5`. Update `currentEnd` to `8`, `chainCount` to `3`.
4. **Result:** `chainCount = 3` indicates the maximum number of pairs that can be chained.


Input: pairs = [[7,8], [5,6], [1,2], [3,5], [4,5], [2,3]]
Pairs after sorting.
1, 2
2, 3
3, 5
4, 5
5, 6
7, 8
chainCount = 0
currentEnd = -Infinity
chainCount stores the number of included pairs in the chain, and
currentEnd stores the second element of the last pair in chain.
chainCount = 1
currentEnd = 2
pair[0] = 1 > currentEnd = -Infinity.
So, do chainCount++, and update currentEnd = 2.
chainCount = 1
currentEnd = 2
pair[0] = 2 !> currentEnd = 2.
So, skip the pair.
chainCount = 2
currentEnd = 5
pair[0] = 3 > currentEnd = 2.
So, do chainCount++, and update currentEnd = 5.
chainCount = 2
currentEnd = 5
pair[0] = 4 !> currentEnd = 5.
So, skip the pair.
chainCount = 2
currentEnd = 5
pair[0] = 5 !> currentEnd = 5.
So, skip the pair.
chainCount = 3
currentEnd = 8
pair[0] = 7 > currentEnd = 5.
So, do chainCount++, and update currentEnd = 8.
Final chain is [1, 2] -> [3, 5] -> [7, 8] and its length is 3.

Code

Here is the code for this algorithm:



```java
import java.util.Arrays;

public class Solution {

    public int findLongestChain(int[][] pairs) {
        // Sort pairs based on their second element
        Arrays.sort(pairs, (a, b) -> a[1] - b[1]);

        // Initialize variables
        int currentEnd = Integer.MIN_VALUE;
        int chainCount = 0;

        // Iterate through the pairs
        for (int[] pair : pairs) {
            if (pair[0] > currentEnd) {
                // Update currentEnd and increment chainCount
                currentEnd = pair[1];
                chainCount++;
            }
        }

```

You must run your code first

Complexity Analysis

- **Time Complexity**: The code takes (O(n log n)) time to run, mainly due to sorting the input pairs.
- **Space Complexity**: It uses (O(n)) additional space due to the input array of pairs.

← Previous
Maximum Length Of Pair Chain (Medium)

Next →
Minimum Add To Make Parentheses Valid (Medium)

☐ Mark as Completed